University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

# Versuchsaufbau und Methodik

Wie die Projektarbeit „Wie feingranular muss Observability sein?“ ihre Ergebnisse überprüfbar gewinnt

Begleitdokument zum Exposé

Wintersemester 2026/27

**Autor:** Erlind Sejdiu
**Matrikelnummer:** 106357

Stand: 28. Oktober 2026

# Inhaltsverzeichnis


**1 Auf einen Blick** **1**

**2 Wie messen andere Arbeiten?** **1**

2.1 Nõu et al. 2025: Kosten von Tracing . . . . . 2

2.2 Borges et al. 2024 (OXN): Sichtbarkeit von Fehlern . . . . . 2

2.3 Zhou et al. (TrainTicket): Fehlersuche durch Menschen . . . . . 2

2.4 Pham et al. (RCAEval): automatische Ursachenanalyse . . . . . 3

2.5 Diese Arbeit im Vergleich . . . . . 3

**3 Methodik und Versuchsaufbau** **3**

**4 Methodik: Was wird verändert?** **4**

**5 Versuchsaufbau in diesem Beispiel** **5**

5.1 Gesamtaufbau . . . . . 5

5.2 Last . . . . . 5

5.3 Fehler . . . . . 6

5.4 Messung und Auswertung . . . . . 6

**6 Versuchsplan, Risiken, offene Punkte** **7**

**Literaturverzeichnis** **8**


# Abbildungsverzeichnis


1 Ablauf der Arbeit. Rot: Methodik (wird untersucht und verbessert). Grau: Versuchsaufbau (Messinstrument, bleibt unverändert). . . . . 1

2 Legende für die Abbildungen 3 bis 7 . . . . . 1

3 Messaufbau Nõu et al. . . . . 2

4 Messaufbau OXN . . . . . 2

5 Messaufbau TrainTicket . . . . . 2

6 Messaufbau RCAEval . . . . . 3

7 Messaufbau dieser Arbeit . . . . . 3

8 Granularitätsstufen am Beispiel . . . . . 4

9 Vier Erfassungsformen . . . . . 4

10 Gesamtaufbau . . . . . 5

11 Offenes und geschlossenes Lastmodell . . . . . 5

12 Kalibrierung und Ablauf eines Messlaufs . . . . . 6

13 Ablauf eines Fehlerversuchs . . . . . 6

14 Zerlegung der Kosten . . . . . 7

15 Ergebnis-Matrix Nutzen . . . . . 7

## 1 Auf einen Blick

**Leitfrage:** Wie fein muss eine .NET-Microservice-Anwendung mit OpenTelemetry instrumentiert werden, damit sich Fehler finden lassen, ohne dass die Instrumentierung zu viel kostet?

**Antwortweg:** Dieselbe Anwendung in vier Stufen instrumentieren, Fehler einbauen und je Stufe *Kosten* und *Nutzen* messen. Der Messaufbau wird aus bestehenden Arbeiten übernommen.

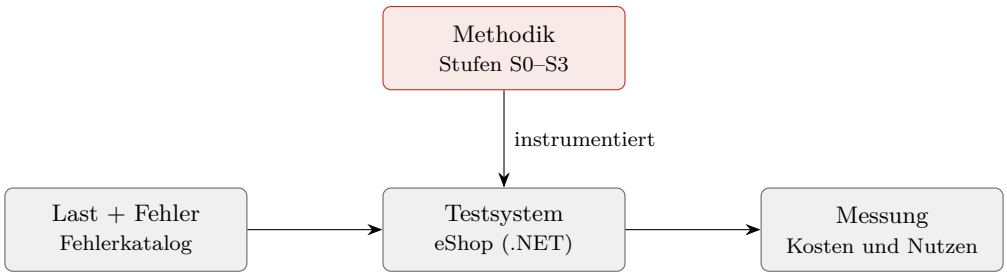



Abbildung 1: Ablauf der Arbeit. Rot: Methodik (wird untersucht und verbessert). Grau: Versuchsaufbau (Messinstrument, bleibt unverändert).

| **Forschungsfrage** | **Erwartung (Hypothese)** |
|---|---|
| FF1 Was kosten die Stufen S1–S3? | H1: Kosten steigen, von S2 auf S3 überproportional. |
| FF2 Woher kommen die Kosten? | H2: Das Versenden der Daten ist der größte Anteil [1]. |
| FF3 Ab welcher Stufe werden Fehler auffindbar? | H3: S2 findet fast so viel wie S3, kostet aber deutlich weniger. |
| FF4 Span, Event, Attribut oder Log? | H4: Events und Attribute sind günstiger als eigene Spans. |

## 2 Wie messen andere Arbeiten?

Alle vier Arbeiten folgen demselben Muster: *Last* erzeugen, *Fehler* einbauen, *eine Stellschraube* verändern, Telemetrie *auswerten*. Die Abbildungen verwenden deshalb dieselbe Darstellung. Leere, gestrichelte Kästen zeigen, was eine Arbeit nicht untersucht.




Abbildung 2: Legende für die Abbildungen 3 bis 7

### 2.1 Nõu et al. 2025: Kosten von Tracing

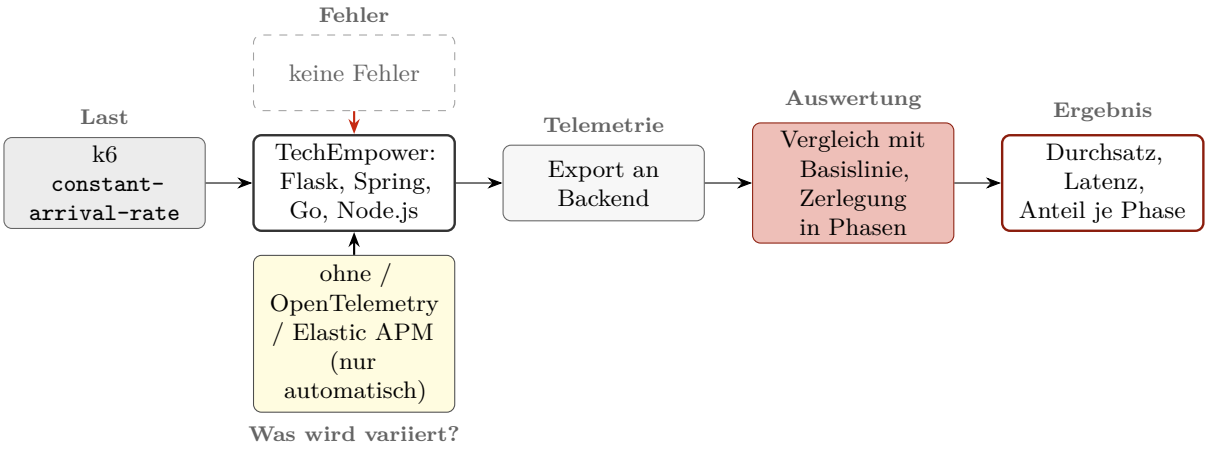



Abbildung 3: Messaufbau von Nõu et al. [1]. Gemessen werden nur Kosten, kein Nutzen.

### 2.2 Borges et al. 2024 (OXN): Sichtbarkeit von Fehlern

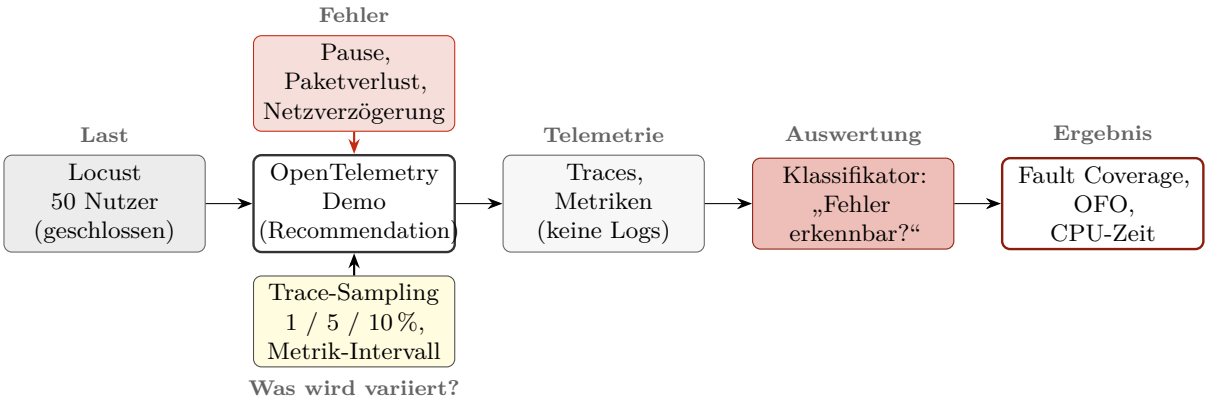



Abbildung 4: Messaufbau von OXN [2]. Variiert wird, *wie viele* Traces aufgezeichnet werden.

### 2.3 Zhou et al. (TrainTicket): Fehlersuche durch Menschen

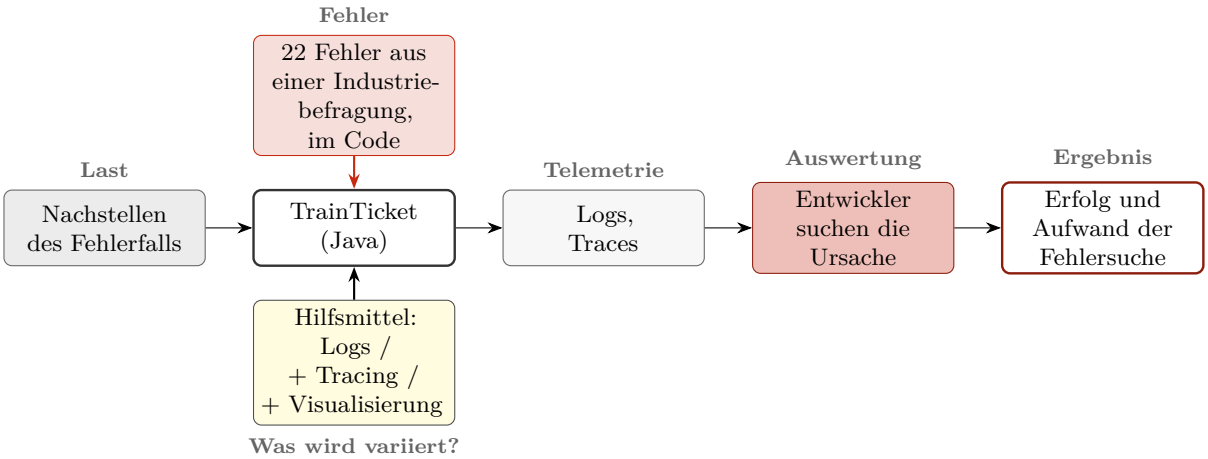



Abbildung 5: Messaufbau von Zhou et al. [3]. Bewertet wird der Nutzen durch Menschen, nicht die Kosten.

### 2.4 Pham et al. (RCAEval): automatische Ursachenanalyse

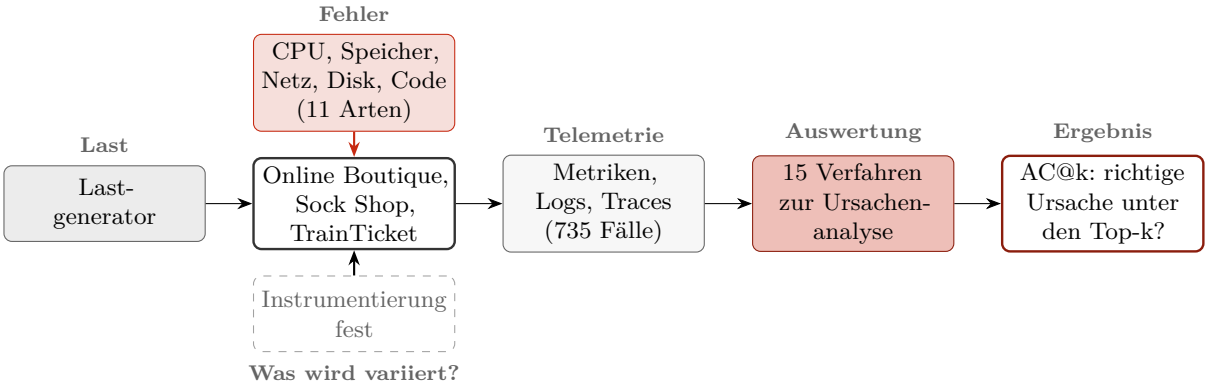



Abbildung 6: Messaufbau von RCAEval [4]. Variiert wird das Auswerteverfahren, nicht die Instrumentierung.

### 2.5 Diese Arbeit im Vergleich

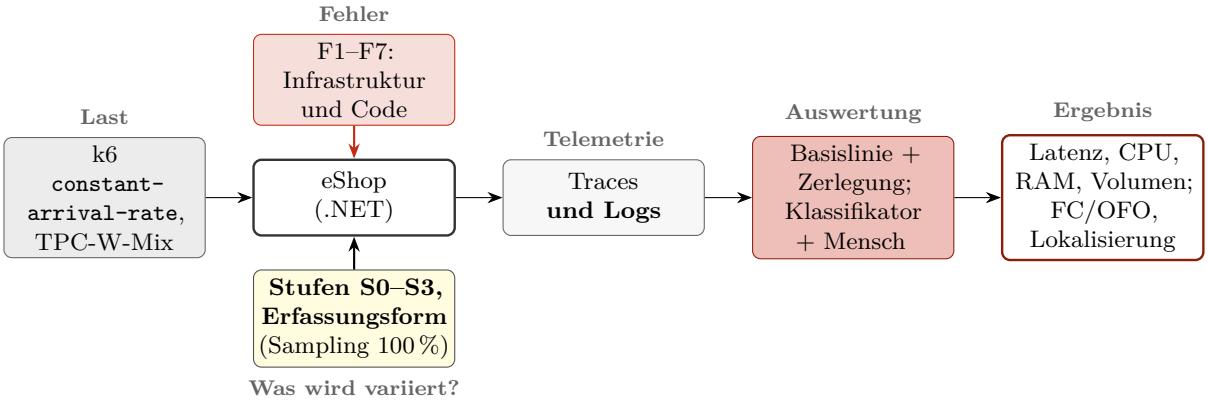



Abbildung 7: Messaufbau dieser Arbeit. Last und Kostenmessung nach Nõu et al., Fehler nach TrainTicket und RCAEval, Sichtbarkeit nach OXN, Fehlersuche durch Menschen nach TrainTicket.

**Abgrenzung:** OXN verändert, *wie viele* Traces aufgezeichnet werden. Diese Arbeit verändert, *was in einem Trace steht*. Nõu et al. messen nur Kosten, TrainTicket und RCAEval nur Nutzen. Diese Arbeit misst beides am selben System.

## 3 Methodik und Versuchsaufbau

| | **Methodik** (wird untersucht) | **Versuchsaufbau** (misst) |
|---|---|---|
| Inhalt | Stufen S0–S3, Erfassungsform, Leitfaden | Testsystem, Last, Fehler, Messung |
| Herkunft | Literatur | bestehende Studien und Benchmarks |
| Änderbar? | ja, nach jeder Iteration | nein, vor der Bewertung eingefroren |

**R1** Der Aufbau wird nicht passend zur Methodik gewählt. **R2** Er wird vor der Bewertung in Git markiert. **R3** Jede Messung ist per Skript wiederholbar.

# 4 Methodik: Was wird verändert?

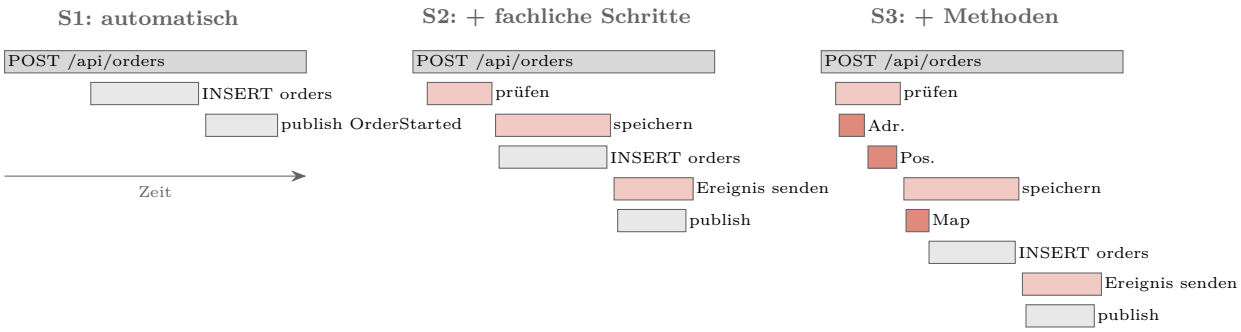



Abbildung 8: Derselbe Vorgang „Bestellung abschicken“ als Trace in den Stufen S1–S3. Grau: automatisch erzeugte Spans, rot: manuell ergänzte Spans. S0 erzeugt keinen Trace.

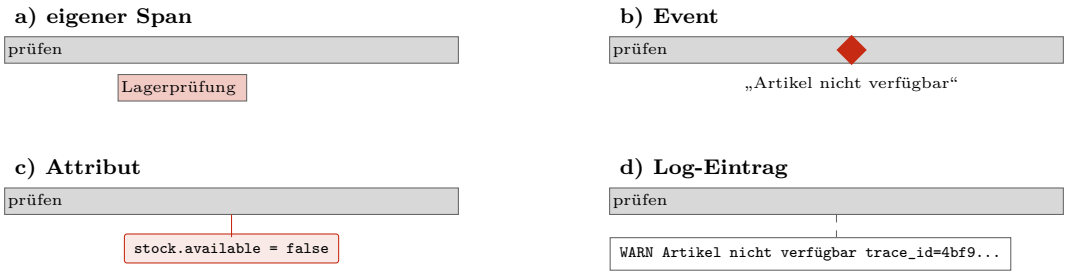



Abbildung 9: Dieselbe Information in vier Formen (Stufe S2). Gemessen wird, was jede Form kostet und ob sie bei der Fehlersuche hilft.

Die Regeln des Leitfadens stammen aus den Semantic Conventions [5], Dapper [6], Canopy [7], Sambasivan et al. [8] und Majors et al. [9]. Jede Regel erhält eine Quelle, ein Beispiel aus eShop und ein Messergebnis.

# 5 Versuchsaufbau in diesem Beispiel

## 5.1 Gesamtaufbau

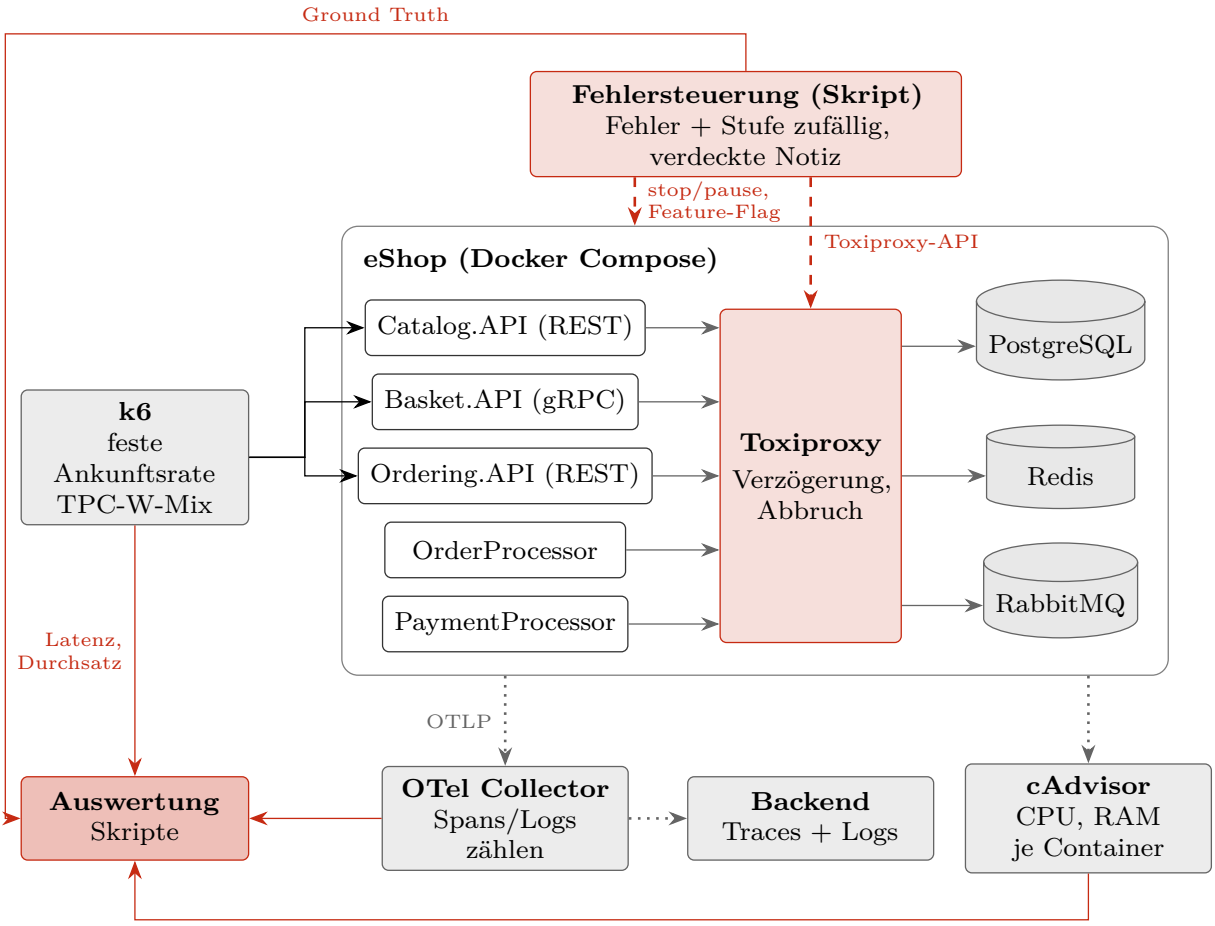



Abbildung 10: Versuchsaufbau mit eShop. Schwarz: Anwendung, rot gestrichelt: Fehler, grau gepunktet: Telemetrie, rot durchgezogen: Messdaten für die Auswertung. Lastgenerator und Messwerkzeuge laufen getrennt von eShop.

## 5.2 Last

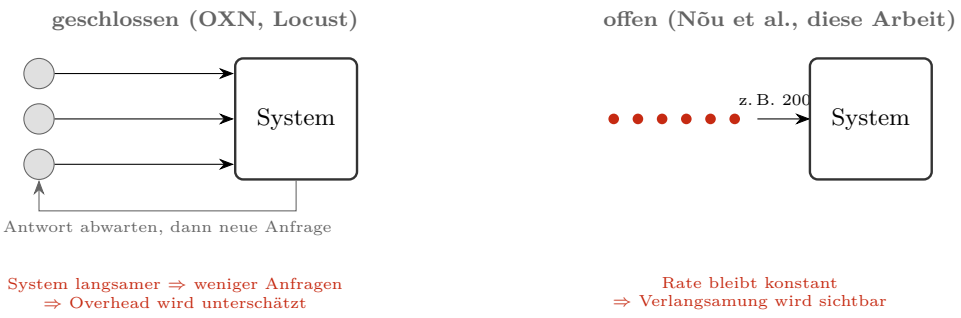



Abbildung 11: Warum eine feste Ankunftsrate verwendet wird [10].

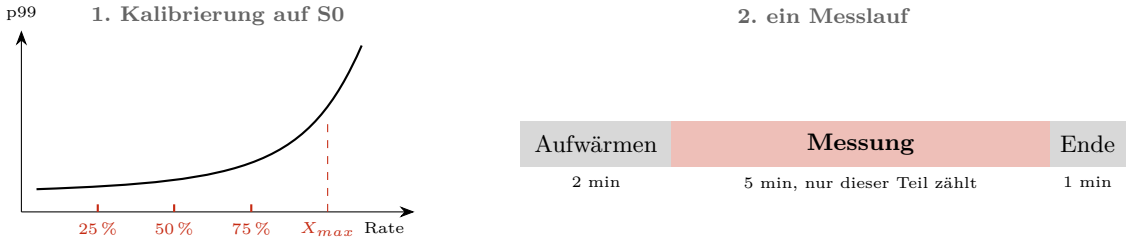



Abbildung 12: Links: maximaler Durchsatz $X_{max}$ ohne Instrumentierung, gemessen wird bei 25, 50 und 75 %. Rechts: Ablauf eines Laufs.

| **Nutzerpfad** | **Aufruf** | **Anteil nach TPC-W [11]** |
|---|---|---|
| Katalog durchsuchen, Artikel ansehen | Catalog.API (REST) | ca. 80 % [prüfen] |
| Warenkorb ändern, bestellen | Basket.API (gRPC), Ordering.API (REST) | ca. 20 % |

## 5.3 Fehler

| **ID** | **Fehler** | **Eingebaut über** | **Vorbild** |
|---|---|---|---|
| F1 | Datenbank langsam | Toxiproxy | RCAEval, OXN |
| F2 | Basket.API ausgefallen | `docker stop` | RCAEval, OXN |
| F3 | OrderProcessor hängt | `docker pause` | OXN |
| F4 | Fehler in Geschäftslogik verschluckt | Feature-Flag | TrainTicket, RCAEval |
| F5 | Dienst rechnet zu lange | Feature-Flag | RCAEval |
| F6 | Redis nicht erreichbar | Toxiproxy | RCAEval |
| F7 | falsche Konfiguration | Umgebungsvariable | TrainTicket |

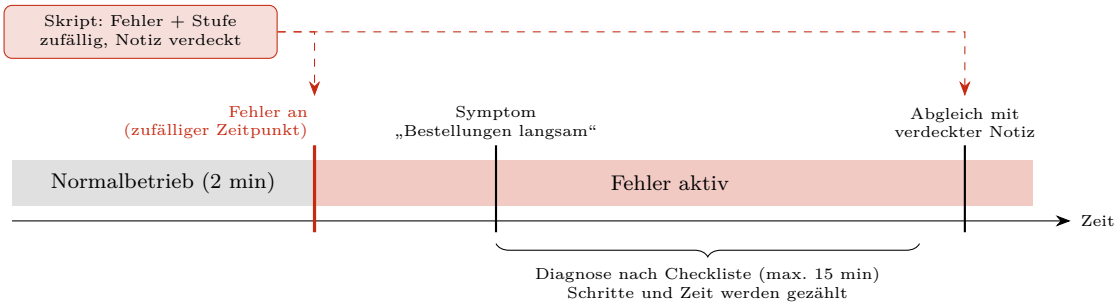



Abbildung 13: Ablauf eines Fehlerversuchs nach dem Muster des Chaos Engineering [12]. Die diagnostizierende Person kennt den aktiven Fehler nicht.

## 5.4 Messung und Auswertung

Kosten werden von außen gemessen (k6, cAdvisor), nie aus den Traces selbst, denn S0 hat keine Traces.

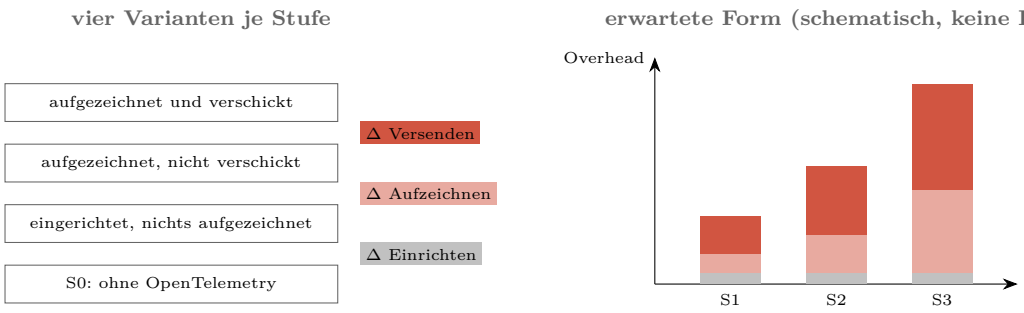



Abbildung 14: Zerlegung der Kosten wie bei Nõu et al. [1]. Die Differenz zweier benachbarter Varianten ergibt den Anteil einer Phase.

| | S1 | S2 | S3 |
|---|---|---|---|
| F1 DB langsam | ja | ja | ja |
| F2 Dienst aus | ja | ja | ja |
| F3 Prozessor hängt | nein | teils | teils |
| F4 Logik verschluckt | nein | ja | ja |
| F5 Rechenlast | teils | ja | ja |
| F6 Redis weg | ja | ja | ja |
| F7 Konfiguration | teils | teils | teils |

Zelle = Ursache gefunden?
Erhoben automatisch (FC/OFO [2])
und durch Menschen ([3]).

Hier: **erwartetes** Muster,
keine Messdaten.

Abbildung 15: Form des Ergebnisses für den Nutzen. Zusammen mit Abbildung 14 zeigt sie je Stufe, was sie kostet und was sie bringt.

| | |
|---|---|
| **Kosten** | Latenz p50/p95/p99 (k6), max. Durchsatz, CPU und RAM (cAdvisor), Spans/Logs/Bytes pro Anfrage (Collector) |
| **Nutzen** | Fault Coverage und OFO (automatisch), gefunden ja/nein, Schritte, Zeit (Mensch) |
| **Gültigkeit** | Fehlerquote $< 1\,\%$, Rate erreicht, Lastgenerator nicht ausgelastet, keine verworfenen Spans |
| **Statistik** | $\geq 10$ Läufe in zufälliger Reihenfolge [im Pilot bestätigen], ein Lauf = ein Datenpunkt, Median mit Konfidenzintervall, Mann-Whitney-U und Cliff's Delta [13], [14] |

## 6 Versuchsplan, Risiken, offene Punkte

| **Versuchsblock** | **Läufe** | **Frage** |
|---|---|---|
| S0–S3 × 3 Lastniveaus × 10 | 120 | FF1 |
| Zerlegung S1–S3 × 2 Zusatzvarianten × 10 (50 % Last) | 60 | FF2 |
| 4 Erfassungsformen × 10 (50 % Last) | 40 | FF4 |
| 7 Fehler × 4 Stufen, mehrfach | [offen] | FF3 |
| **Leistungsmessung gesamt** | **220 ≈ 30 h** | |

| **Risiko** [15] | **Gegenmaßnahme** |
|---|---|
| eigene Bewertung der eigenen Methodik | R1–R3, verdeckte Fehlerauswahl, OXN-Metriken |
| Lerneffekt bei der Diagnose | zufällige Reihenfolge, Varianten je Fehler, zweite Person |
| nur eShop und .NET | ausdrücklich benennen, optional zweites System |
| Messrauschen | eigener Rechner, getrennte CPU-Kerne, Wiederholungen |

**Zur Abstimmung:** Blazor statt Angular in eShop? OXN als Werkzeug wiederverwenden? Analyse-Backend (Jaeger oder Tempo und Loki)? Zweite Person für die Diagnose?

## Literaturverzeichnis


[1] A. Nõu, S. Talluri, A. Iosup und D. Bonetta, „Investigating Performance Overhead of Distributed Tracing in Microservices and Serverless Systems," in *Companion of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE Companion '25)*, 2025, S. 162–166. DOI: 10.1145/3680256.3721316.

[2] M. C. Borges, J. Bauer, S. Werner, M. Gebauer und S. Tai, „Informed and Assessable Observability Design Decisions in Cloud-native Microservice Applications," in *IEEE 21st International Conference on Software Architecture (ICSA 2024)*, 2024. arXiv: 2403.00633.

[3] X. Zhou u. a., „Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study," *IEEE Transactions on Software Engineering*, Jg. 47, Nr. 2, S. 243–260, 2021. DOI: 10.1109/TSE.2018.2887384.

[4] L. Pham, H. Zhang, H. Ha, F. Salim und X. Zhang. „RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data." arXiv: 2412.17015.

[5] OpenTelemetry Authors. „OpenTelemetry Semantic Conventions," besucht am 29. Sep. 2026. Adresse: https://opentelemetry.io/docs/specs/semconv/.

[6] B. H. Sigelman u. a., „Dapper, a Large-Scale Distributed Systems Tracing Infrastructure," Google, Inc., Techn. Ber. dapper-2010-1, 2010.

[7] J. Kaldor u. a., „Canopy: An End-to-End Performance Tracing And Analysis System," in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*, 2017, S. 34–50. DOI: 10.1145/3132747.3132749.

[8] R. R. Sambasivan, R. Fonseca, I. Shafer und G. R. Ganger, „So, You Want to Trace Your Distributed System? Key Design Insights from Years of Practical Experience," Carnegie Mellon University, Parallel Data Laboratory, Techn. Ber. CMU-PDL-14-102, 2014.

[9] C. Majors, L. Fong-Jones und G. Miranda, *Observability Engineering: Achieving Production Excellence*. Sebastopol, CA: O'Reilly Media, 2022.

[10] B. Schroeder, A. Wierman und M. Harchol-Balter, „Open Versus Closed: A Cautionary Tale," in *3rd USENIX Symposium on Networked Systems Design & Implementation (NSDI 06)*, 2006.

[11] Transaction Processing Performance Council. „TPC Benchmark W (Web Commerce) Specification," besucht am 28. Okt. 2026. Adresse: https://www.tpc.org/tpcw/.

[12] A. Basiri u. a., „Chaos Engineering," *IEEE Software*, Jg. 33, Nr. 3, S. 35–41, 2016. DOI: 10.1109/MS.2016.60.

[13] A. Georges, D. Buytaert und L. Eeckhout, „Statistically Rigorous Java Performance Evaluation," in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*, 2007.

[14] T. Kalibera und R. Jones, „Rigorous Benchmarking in Reasonable Time," in *Proceedings of the 2013 International Symposium on Memory Management (ISMM '13)*, 2013.

[15] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell und A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.